

Démosaiçage d'une image

COQUARD-MOLINARI Julie 12200076 - KRSINAR Xavier 12104784

1 - Analyse d'une image couleur

Pour afficher l'image couleur, nous utilisons le morceau de code ci-dessous. On obtient la figure suivante.

```
I = double(imread('ImagesOriginales/19.bmp'))/255;
```

`imread` permet de lire l'image sélectionnée

`figure` fait apparaître la fenêtre graphique

`image(I)` fait apparaître l'image sur la fenêtre graphique

`axis('image')` dimensionne correctement l'image.

`axis('off')` fait disparaître les axes de l'image

`size(I)` donne les dimensions de l'image I, enregistrée sous la variable `ans` pour "answer".

`ans = [1024 x 768 x 3]` donc **la matrice I est de dimension 1024 x 768 x 3**.

(La matrice à donc 1024*768 pixels. Le 3 correspond aux trois couleurs rouge, bleu et vert).



Pour afficher la **composante rouge** de l'image nous utilisons le code suivant :

```
image(cat(3,I(:,:,1),zeros(size(I,1),size(I,2)),zeros(size(I,1),size(I,2))))
```

En modifiant la partie `I(:,:,1)` nous pouvons obtenir la composante bleue ou verte :

-Bleue : `image(cat(3,zeros(size(I,1),size(I,2)),zeros(size(I,1),size(I,2)),I(:,:,3)))`

-Verte : `image(cat(3,zeros(size(I,1),size(I,2)),I(:,:,2),zeros(size(I,1),size(I,2))))`

Ce qui donne à l'affichage (avec les mêmes lignes de code que pour l'image I) les figures ci-dessous.

`cat(3, , , )` permet de concaténer trois matrices. Ici, cette matrice représente une image couleurs par la concaténation des trois matrices représentant les trois composantes R,V et B.

Pour afficher seulement la composante rouge, on prend pour composante rouge la composante rouge de I avec l'instruction `I(:,:,1)`, et pour les composantes verte et bleue, on prend les matrices de zéros.

C'est la même logique qui est suivie pour afficher la composante bleue ou la composante verte.



Nous avons ensuite calculé **l'image en luminance** en additionnant les trois composantes de couleur de l'image initiale que nous avons par la suite divisé par 3.

Pour l'affichage, il faut que l'image soit en niveaux de gris.

colormap permet de définir la palette de couleur de l'image et gray permet de choisir comme palette les niveaux de gris.

Il faut donc que les valeurs soient comprises entre 0 et 255, d'où la nécessité de faire `image(L*255)`

On obtient alors la figure suivante.

```
L=sum(I,3)/3;
figure
image (L*255);
colormap(gray(256))
axis('image')
axis('off')
```



Pour l'**image en chrominance**, on soustrait l'image de luminance à l'image originale.

Ainsi, la matrice de la composante rouge de la chrominance est égale à la matrice de composante rouge de l'originale moins la matrice de la luminance. Idem pour la composante verte et la composante bleue.

La matrice de la chrominance est alors la concaténation de ces trois matrices rouge, verte et bleue.

Pour l'affichage, il est nécessaire d'augmenter les valeurs dans toutes les composantes avec `image(Chr+0.5)`, sans quoi on a une image trop sombre pour être visible.

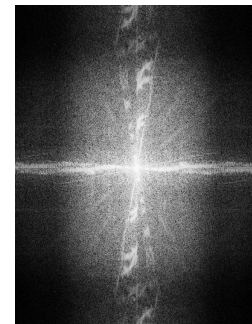
On obtient alors la figure suivante.

```
Chr = cat(3,I(:,:,1)-L, I(:,:,2)-L, I(:,:,3)-L);
figure
image(Chr+0.5)
axis('image')
axis('off')
```



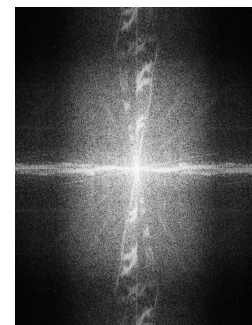
Ensuite nous avons calculé et affiché les modules de la **transformée de Fourier de l'image de luminance** grâce au code suivant.

```
fL = abs(fftshift(fft2(L)));
fLnorm = fL./(fL+mean(fL(:)));
figure
image(255*fLnorm)
axis('image')
axis('off')
colormap(gray(256))
```



Pour calculer la **transformée de Fourier spatiale de l'image du canal rouge**, on utilise la même méthode mais en utilisant la fonction `abs(fftshift(fft2(R)))` sur la matrice du canal rouge $R = I(:,:,1)$ plutôt que sur la matrice de la luminance L . On obtient alors la figure suivante.

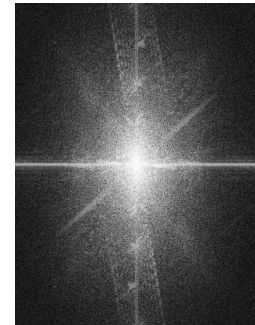
```
R = I(:,:,1);
fR = abs(fftshift(fft2(R)));
fRnorm = fR./(fR+mean(fR(:)));
figure
image(255*fRnorm)
axis('image')
axis('off')
colormap(gray(256))
```



Pour calculer la **transformée de Fourier spatiale du premier canal de la chrominance**, on utilise la même méthode mais en utilisant la fonction `abs(fftshift(fft2( )))` sur la matrice du premier canal de la chrominance `Chr_R = Chr(:, :, 1)`.

On obtient alors la figure suivante.

```
Chr_R= Chr(:, :, 1);
fChr_R = abs(fftshift(fft2(Chr_R)));
fChr_Rnorm = fChr_R./(fChr_R+mean(fChr_R(:)));
figure
image(255*fChr_Rnorm)
axis('image')
axis('off')
colormap(gray(256))
```



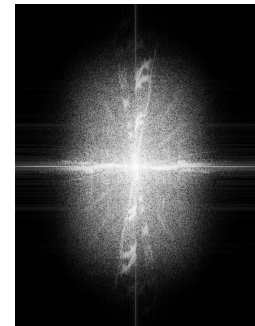
On utilise la même méthode pour la **transformée de Fourier spatiale de l'image en luminance filtrée**, en utilisant la fonction `abs(fftshift(fft2( )))` sur la matrice de l'image en luminance filtrée `L1`.

Avant cela, il faut créer l'image `L1` avec le code suivant :

```
filtre = [1 2 1; 2 4 2; 1 2 1]/16;
L1 = conv2(L, filtre, 'same');
```

On peut alors faire la transformée de Fourier :

```
fL1 = abs(fftshift(fft2(L1)));
Llnorm = fL1./(fL1+mean(fL1(:)));
figure
image(255*Llnorm)
axis('image')
axis('off')
colormap(gray(256))
```



On obtient alors la figure ci-dessus.

On remarque que la transformée de Fourier spatiale de l'image en luminance filtrée est plus sombre que les autres sur les bords. La taille du spectre de Fourier de l'image a été réduite grâce à un filtre passe bas appliqué sur chaque canal de l'image.

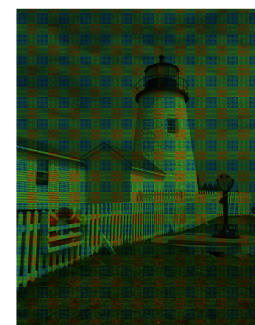
On remarque que, sur l'image de la FFT de la chrominance, les bords sont plus lumineux. Pour la FFT de la luminance, les bords sont moins lumineux.

De plus, le trait vertical et horizontal est plus visible dans la FFT de la chrominance.

2 - Création de la mosaïque

Le code ci-dessous permet de créer et d'afficher l'**image mosaïque**.

```
[H W P] = size(I);
[x y] = meshgrid(1:W, 1:H);
mR = (1+cos(pi*x)).*(1+cos(pi*y))/4;
mG = (1-cos(pi*x)).*cos(pi*y)/2;
mB = (1-cos(pi*x)).*(1-cos(pi*y))/4;
Imos = cat(3, I(:, :, 1).*mR, I(:, :, 2).*mG, I(:, :, 3).*mB);
figure
image(Imos)
axis('image')
axis('off')
```



On crée la grille avec `meshgrid` puis on utilise `mR`, `mG`, `mB` afin de colorier cette grille avec les couleurs rouge, verte et bleue.

Ensuite, on concatène (grâce à l'instruction `cat(3, , , )`) les grilles qui correspondent à chaque couleur dans `Imos`.

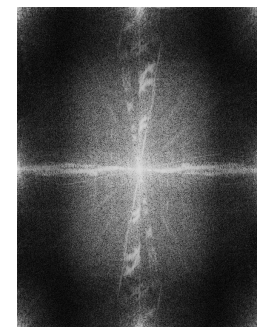
Nous affichons ensuite l'**image mosaïque en niveau de gris** grâce à la même méthode que pour l'image en luminance. On obtient la figure suivante.

```
ImosNB=sum(Imos,3);
figure
image(ImosNB*255)
axis('image')
axis('off')
colormap(gray(256))
```



Nous affichons ensuite le **module de la transformée de Fourier de l'image mosaïque en niveau de gris**. On utilise le code suivant :

```
fMos = abs(fftshift(fft2(ImosNB)));
fMosnorm = fMos./(fMos+mean(fMos(:)));
figure
image(225*fMosnorm)
axis('image')
axis('off')
colormap(gray(256))
```



On utilise la fonction `abs(fftshift(fft2))` qui permet de calculer la transformée de Fourier et nous l'appliquons sur la mosaïque en niveau de gris (`ImosNB`).

Algorithme de démosaïçage par luminance/chrominance

Nous avons utilisé deux filtres de luminance et d'interpolation des couleurs (`flum` et `flum2`) afin d'effectuer démosaïçage de l'image en mosaïque calculée précédemment

Pour le démosaïçage, on crée `LMos` grâce au filtre de luminance `flum` (on aurait aussi pu choisir `flum2`). On obtient alors une image en luminance.

On calcule ensuite l'image de luminance

```
LMos = conv2(Imos_NB,flum,'same');
figure
image(255*LMos)
axis('image')
axis('off')
colormap(gray(256))
```

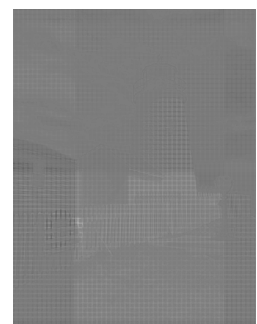


pour mettre le filtre de luminance nous utilisons

```
LMos = conv2(Imos_NB,flum,'same');
```

On soustrait l'image mosaïque par l'image en luminance que l'on vient de calculer pour obtenir la **chrominance multiplexée**.

```
ChrMos = Imos_NB - LMos;
figure
image((ChrMos+0.5)*255);
image(ChrMos*255);
axis('image')
axis('off')
```



Pour obtenir la chrominance à partir de la chrominance multiplexée, on effectue un démultiplexage de la chrominance, puis on interpole cette image.

```
ChrDem = cat(3, ChrMos.*mR, ChrMos.*mG, ChrMos.*mB);
figure
image(ChrDem+0.5)
axis('image')
axis('off')
colormap(gray(256))
fRB = [1 2 1; 2 4 2; 1 2 1]/4;
fG = [0 1 0; 1 4 1; 0 1 0]/4;
```



Interpolation de la chrominance :

```
ChrInt = cat(3, conv2(ChrDem(:,:,1), fRB, 'same'), conv2(
ChrDem(:,:,2), fG, 'same'), conv2(ChrDem(:,:,3), fRB, 'same')));
figure
image(ChrInt+0.5);
axis('image')
axis('off')
colormap(gray(256))
```

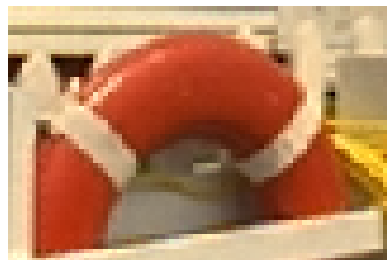
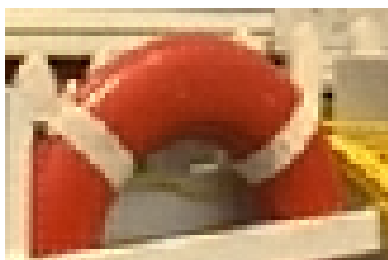


Ici, $\text{ChrDem}(:,:,1)$ représente la chrominance démosaïcée pour la composante rouge de l'image. De même pour la composante rouge, bleue et verte.

Ensuite il suffit de combiner l'image en luminance et en chrominance pour reconstruire l'image.

```
I2 = cat(3, LMos+ChrInt(:,:,1), LMos+ChrInt(:,:,2), LMos+ChrInt(:,:,3));
figure
image(I2);
axis('image')
axis('off')
```

Remarque : Les deux filtres flum et flum2 donnent des résultats légèrement différents (flum à gauche et flum2 à droite, ci-dessous). On remarque une légère différence sur ces deux images au niveau de la couleur.



Références :

D. Alleysson, S. Susstrunk and J. Herault, "Linear demosaicing inspired by the human visual system," in *IEEE Transactions on Image Processing*, vol. 14, no. 4, pp. 439-449, April 2005, doi: 10.1109/TIP.2004.841200.